Executions · Khalid

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Khalid 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

1. Retrieve complete information for any academic member using their ID, including their role(s) and access scopes.
   Signature:
   Name: ViewMemberInfo.
   Type: Function.
   Input: @MemberID int.
   Output: Row containing all member details, role(s) and access scopes.

2. Add a new academic member to the system.
   Signature:
   Name: AddAcademicMember.
   Type: Stored procedure.
   Input: @FullName varchar(150), @UniversityIDNo varchar(50), @Email varchar(150), @Phone varchar(50), @AcademicRank varchar(50), @EmploymentStatus varchar(50), @AccountStatus varchar(50), @FacultyID int.
   Output: Confirmation message or new @MemberID int.

3. Update an academic member's contact details.
   Signature:
   Name: UpdateMemberContactInfo.
   Type: Stored procedure.
   Input: @MemberID int, @Email varchar(150), @Phone varchar(50).
   Output: Confirmation message.

4. Activate, suspend, or deactivate a member's account.
   Signature:

Name: UpdateAccountStatus.
Type: Stored procedure.
Input: @MemberID int, @AccountStatus varchar(50).
Output: Confirmation message.

5. Register an existing member as an Academic Instructor.
   Signature:
   Name: AssignInstructorRole.
   Type: Stored procedure.
   Input: @MemberID int, @ResearchArea varchar(150), @OfficeLocation varchar(50), @TeachingLoad int.
   Output: Confirmation message.
6. Register an existing member as a Teaching Assistant under a supervising instructor.
   Signature:
   Name: AssignTeachingAssistantRole.
   Type: Stored procedure.
   Input: @MemberID int, @AssistantLevel varchar(50), @SupervisorID int.
   Output: Confirmation message.
7. Appoint a member as the Dean of a faculty (the faculty can only have one dean at a time).
   Signature:
   Name: AssignDean.
   Type: Stored procedure.
   Input: @MemberID int, @FacultyID int, @ApprovalAuthorityScope varchar(150).
   Output: Confirmation message.
8. Register an existing member as part of University Management.
   Signature:
   Name: AssignManagementRole.
   Type: Stored procedure.
   Input: @MemberID int, @ManagementTitle varchar(150), @DecisionAuthorityLevel varchar(50), @GovernanceScope varchar(150).
   Output: Confirmation message.
9. Register an existing member as a Curriculum Reviewer and record their committee membership.
   Signature:
   Name: AssignReviewerRole.
   Type: Stored procedure.
   Input: @MemberID int, @ReviewDomain varchar(150), @ReviewAuthorityLevel varchar(50), @CommitteeMembership varchar(150).
   Output: Confirmation message.
10. Grant an access scope to a member (e.g., “Submit Proposals”, “Approve Initiatives”).
    Signature:
    Name: GrantAccessScope.
    Type: Stored procedure.
    Input: @MemberID int, @AccessScope varchar(50).
    Output: Confirmation message.
11. Revoke an access scope from a member.
    Signature:
    Name: RevokeAccessScope.
    Type: Stored procedure.

Input: @MemberID int, @AccessScope varchar(50).
Output: Confirmation message.

12. Add a new faculty.
Signature:
Name: AddFaculty.
Type: Stored procedure.
Input: @Name varchar(150), @Description varchar(1000), @EstablishmentDate date, @DeanID int.
Output: Confirmation message or new @FacultyID int.

17. Record the semesters during which a dean is in office.
Signature:
Name: RecordDeanTerm.
Type: Stored procedure.
Input: @DeanID int, @SemesterID int.
Output: Confirmation message.

21. View all platform activity recorded in the audit log within a date range.
Signature:
Name: ViewAuditLog.
Type: Function.
Input: @StartDate date, @EndDate date.
Output: Table showing action type, actor, affected item, timestamp and change summary.

22. Retrieve a summary of academic member distribution across faculties and roles.
Signature:
Name: GetFacultyMemberStats.
Type: Function.
Input: Nothing.
Output: Table showing faculty, role and number of members.

3 As a Teaching Assistant, I should be able to:

2. View my supervising instructor’s details.
Signature:
Name: ViewMySupervisor.
Type: Function.
Input: @TAID int.
Output: Row containing supervisor name, email and office location.

6 As an University Management member, I should be able to:

11. Send an announcement to all members who hold a specific role.
Signature:
Name: BroadcastNotification.
Type: Stored procedure.
Input: @MessageContent varchar(1000), @Urgency varchar(50), @Category varchar(50), @TargetRole varchar(50).
Output: Confirmation message.

12. View the complete audit trail of a proposal, approval, initiative, assessment or piece of evidence.
Signature:
Name: ViewItemAuditTrail.

Type: Function.
Input: @ItemType varchar(20), @ItemID int.
Output: Table showing action type, actor, timestamp and change summary.

7 As an Academic Member, I should be able to:

1. View my own profile.
   Signature:
   Name: ViewMyProfile.
   Type: Function.
   Input: @MemberID int.
   Output: Row containing my details, role(s) and access scopes.

2. View my notifications, optionally showing only unread ones.
   Signature:
   Name: ViewMyNotifications.
   Type: Function.
   Input: @MemberID int, @UnreadOnly bit.
   Output: Table of notifications ordered by timestamp.

3. Count my unread notifications.
   Signature:
   Name: GetUnreadNotificationCount.
   Type: Function.
   Input: @MemberID int.
   Output: Number of unread notifications int.

4. Mark a notification as read.
   Signature:
   Name: MarkNotificationRead.
   Type: Stored procedure.
   Input: @NotificationID int, @MemberID int.
   Output: Confirmation message.

12. Receive a notification when a decision is made on a proposal or initiative I submitted.
    Signature:
    Name: NotifyApprovalDecision.
    Type: Stored procedure.
    Input: @MemberID int, @ItemType varchar(20), @ItemID int, @Decision varchar(50).
    Output: Notification message.